

Guia do Código, Linha a Linha

Como cada parte funciona — e como ler cada gráfico

Material de apoio — Seminário de Algoritmos Bioinspirados

Lucas Rivetti Ian Nunes

Junho de 2026

Como usar este guia. Para cada arquivo do projeto, explicamos o que ele faz, mostramos os trechos de código mais importantes e os traduzimos em português simples. No fim, há uma seção ensinando **como interpretar cada gráfico** gerado. Não é preciso saber programar para acompanhar: foque nos boxes azuis (a “intuição”).

Sumário

1	Visão geral: os arquivos e o fluxo	2
2	problema.py — o problema	2
2.1	A classe <code>Instancia</code>	2
2.2	<code>gerar_instancia</code> — cria um cenário	2
2.3	<code>avaliar</code> — calcula os dois objetivos	3
2.4	<code>first_fit</code> e <code>first_fit_decreasing</code>	3
3	metricas.py — comparar fronteiras	3
3.1	<code>domina</code> — quem é melhor	3
3.2	<code>frente_nao_dominada</code> — acha a fronteira	3
3.3	As quatro métricas	4
4	vmpacs.py — a colônia de formigas	4
4.1	<code>construir_solucao</code> — uma formiga monta uma alocação	4
4.2	<code>vmpacs</code> — o laço principal	4
5	nsga2.py — o algoritmo genético	5
5.1	<code>ordenacao_nao_dominada</code> — separa em frentes	5
5.2	<code>distancia_multidao</code> — mantém diversidade	5
5.3	<code>crossover_ox</code> e <code>mutacao_swap</code> — gerar filhos	5
6	referencia.py — a fronteira “melhor conhecida”	5
7	experimentos.py — rodar tudo	5
8	Como ler cada gráfico	6
8.1	Gráficos de barras de energia, desperdício e servidores	6
8.2	Gráficos de Hipervolume e IGD	6
8.3	Gráfico das fronteiras de Pareto	6

1 Visão geral: os arquivos e o fluxo

O projeto tem 6 módulos Python, cada um com uma responsabilidade:

Arquivo	Responsabilidade
problema.py	Define o problema: gera instâncias, avalia energia e desperdício.
metricas.py	Dominância, fronteira de Pareto e as métricas (HV, IGD, ONVG, SP).
vmpacs.py	O algoritmo de colônia de formigas (VMPACS).
nsga2.py	O algoritmo genético NSGA-II (comparação).
referencia.py	A fronteira de Pareto “melhor conhecida” (referência).
experimentos.py	Roda tudo e gera as tabelas e figuras.

Fluxo: `experimentos.py` pede uma instância a `problema.py`, roda `vmpacs.py` e `nsga2.py` sobre ela, mede a qualidade com `metricas.py`, calcula a referência com `referencia.py` e salva tabelas/-figuras.

A “solução” em uma frase. Tudo gira em torno de um vetor `aloc` de tamanho n : `aloc[i]` é o servidor que recebe a VM i . Por exemplo, `aloc = [0, 0, 1, 2, 1]` significa: VMs 0 e 1 no servidor 0; VM 2 no servidor 1; VM 3 no servidor 2; VM 4 no servidor 1.

2 problema.py — o problema

2.1 A classe Instancia

Guarda os dados de um cenário: demandas das VMs, número de servidores, limites e constantes de energia.

```
class Instancia:
    def __init__(self, Rp, Rm, m, Tp=0.9, Tm=0.9, P_idle=162.0, P_busy=215.0, eps=1e-4):
        self.Rp = np.asarray(Rp)      # demanda de CPU de cada VM
        self.Rm = np.asarray(Rm)      # demanda de memoria de cada VM
        self.n = len(self.Rp)         # numero de VMs
        self.m = m                    # numero de servidores disponiveis
        self.Tp = Tp; self.Tm = Tm    # limites de CPU e memoria (90%)
        self.P_idle = P_idle; self.P_busy = P_busy    # potencias (W)
        self.total_cpu = float(np.sum(self.Rp))      # CPU total (constante)
```

`total_cpu` é a soma das CPUs de todas as VMs. Ela é **constante** (todas as VMs rodam) e, como vimos, é a chave para calcular a energia de forma exata.

2.2 gerar_instancia — cria um cenário

Reproduz o gerador correlacionado do artigo: sorteia CPU e memória de cada VM e, conforme o parâmetro P , cria correlação entre elas.

```
for i in range(n):
    Rp[i] = rng.random() * (2 * Rp_ref)    # CPU em [0, 2*Rp_ref)
    Rm[i] = rng.random() * Rm_ref          # memoria em [0, Rm_ref)
    r = rng.random()
    if (r < P and Rp[i] >= Rp_ref) or (r >= P and Rp[i] < Rp_ref):
        Rm[i] = Rm[i] + Rm_ref             # cria a correlacao
```

O que P faz: $P \approx 0$ gera correlação *negativa* (CPU alta combina com memória baixa); $P \approx 1$ gera correlação *positiva* (alta com alta); $P \approx 0,5$, quase nenhuma. Isso muda o quão difícil é equilibrar os servidores.

2.3 avaliar — calcula os dois objetivos

O coração da modelagem. Soma CPU e memória por servidor e calcula energia e desperdício.

```
np.add.at(usado_cpu, aloc, inst.Rp) # soma a CPU de cada VM no seu servidor
np.add.at(usado_mem, aloc, inst.Rm) # idem para memoria
em_uso = usado_cpu > 0 # servidores que receberam VMs
k = int(np.count_nonzero(em_uso)) # quantos servidores estao ligados

# Objetivo 1 (energia, forma exata): depende so do numero de servidores k
energia = (inst.P_busy - inst.P_idle) * inst.total_cpu + inst.P_idle * k

# Objetivo 2 (desperdicio): soma de |sobra_cpu - sobra_mem| / (usado)
Lp = inst.Tp - usado_cpu[em_uso]; Lm = inst.Tm - usado_mem[em_uso]
W_j = (np.abs(Lp - Lm) + inst.eps) / (usado_cpu[em_uso] + usado_mem[em_uso])
desperdicio = float(np.sum(W_j))
```

`np.add.at(usado_cpu, aloc, inst.Rp)` é um truque do NumPy: para cada VM i , soma $Rp[i]$ na posição $aloc[i]$ de `usado_cpu`. No fim, `usado_cpu[j]` = CPU total usada no servidor j . A energia usa a fórmula exata $(P_{busy} - P_{idle}) \cdot \text{CPU total} + P_{idle} \cdot k$, sem ruído numérico. O desperdício é a soma dos W_j dos servidores em uso.

2.4 first_fit e first_fit_decreasing

Heurística clássica de empacotamento: coloca cada VM no *primeiro* servidor onde ela cabe; se não couber em nenhum, abre um novo.

```
for i in ordem: # percorre as VMs na ordem dada
    for j in range(len(cap_cpu)): # tenta nos servidores ja abertos
        if cabe(cap_cpu[j], cap_mem[j], inst.Rp[i], inst.Rm[i], inst):
            cap_cpu[j] += inst.Rp[i]; cap_mem[j] += inst.Rm[i]
            aloc[i] = j; colocado = True; break
    if not colocado: # nao coube -> abre servidor novo
        aloc[i] = len(cap_cpu); cap_cpu.append(inst.Rp[i]); cap_mem.append(inst.Rm[i])
```

O `first_fit_decreasing` (FFD) é o mesmo, mas ordena as VMs da maior demanda para a menor antes — costuma usar menos servidores. O FFD gera a solução inicial S_0 do VMPACS e serve de *decodificador* do NSGA-II.

3 metricas.py — comparar fronteiras

3.1 domina — quem é melhor

```
def domina(a, b):
    # 'a' domina 'b' se nao e pior em nada E e melhor em algo (minimizacao)
    return np.all(a <= b) and np.any(a < b)
```

Tradução direta da definição de Pareto: a domina b se $a \leq b$ em todos os objetivos e estritamente menor em pelo menos um.

3.2 frente_nao_dominada — acha a fronteira

Marca como “dominada” toda solução que alguém domina; sobram as não-dominadas.

```
for i in range(n):
    for j in range(n):
        if i != j and nao_dom[j] and domina(pontos[j], pontos[i]):
```

```
nao_dom[i] = False; break # alguem domina i -> i sai da fronteira
```

3.3 As quatro métricas

- `onvg(frente) = len(frente)`: número de soluções (maior = melhor).
- `spacing(frente)`: desvio-padrão das distâncias ao vizinho mais próximo (menor = mais uniforme).
- `hipervolume(frente, ref)`: área dominada pela fronteira em relação a um ponto de referência (maior = melhor). Para 2 objetivos, soma faixas:

```
f = f[np.argsort(f[:, 0])] # ordena por energia
hv = 0.0; prev_y = ref[1]
for x, y in f:
    hv += (ref[0] - x) * (prev_y - y) # area de cada retangulo
    prev_y = y
```

- `igd(frente, referencia)`: distância média da fronteira de *referência* até a fronteira obtida (menor = melhor; mede o quão perto a solução chegou da ideal).

4 vmpacs.py — a colônia de formigas

4.1 construir_solucão — uma formiga monta uma alocação

É o trecho mais importante. A formiga abre um servidor e o enche escolhendo VMs; quando nada mais cabe, abre o próximo.

```
while disp.any(): # enquanto houver VMs nao alocadas
    host_cpu = 0.0; host_mem = 0.0 # abre um servidor novo (vazio)
    while True:
        # candidatas: VMs disponiveis que CABEM no servidor atual
        cand = np.where(disp & (host_cpu+Rp <= Tp) & (host_mem+Rm <= Tm))[0]
        if len(cand) == 0: break # nada mais cabe -> fecha o servidor
        # HEURISTICA: contribuicao parcial aos 2 objetivos (energia e desperdicio)
        eta = eta1 + eta2
        # combina FEROMONIO + HEURISTICA
        score = alpha * tau[cand, host] + (1 - alpha) * eta
        # REGRA: q<=q0 escolhe o melhor (intensifica); senao sorteia (explora)
        if rng.random() <= q0: escolhida = cand[np.argmax(score)]
        else: escolhida = rng.choice(cand, p=score/score.sum())
        # aplica o movimento e a ATUALIZACAO LOCAL do feromonio
        aloc[escolhida] = host; disp[escolhida] = False
        tau[escolhida, host] = (1-rho_l)*tau[escolhida, host] + rho_l*tau0
```

Linha a linha: (1) acha as VMs que cabem; (2) calcula a heurística η (quanto cada VM ajuda nos dois objetivos); (3) soma feromônio e heurística no `score`; (4) com prob. `q0` pega a de maior `score` (intensifica), senão sorteia proporcional (explora); (5) coloca a VM e evapora um pouco do feromônio daquele movimento (atualização local).

4.2 vmpacs — o laço principal

```
S0 = first_fit_decreasing(inst) # solucao inicial (FFD)
tau0 = 1.0 / (n * (P_S0 + W_S0)) # feromonio inicial
tau = np.full((n, m), tau0)
for t in range(1, M + 1): # M iteracoes
    for _ in range(NA): # NA formigas constroem solucoes
        aloc = construir_solucão(...)
```

```

    atualizar_arquivo(arquivo, objs, aloc, t) # guarda nao-dominadas
for item in arquivo: # ATUALIZACAO GLOBAL (so as de Pareto)
    reforco = rho_g * lam / (P_S + W_S)
    tau[i, j] = (1-rho_g)*tau[i, j] + reforco

```

A cada iteração, todas as formigas constroem soluções; as não-dominadas entram no **arquivo** (a fronteira de Pareto); depois, só essas soluções *reforçam* o feromônio dos seus movimentos. Assim a colônia “aprende”.

5 nsga2.py — o algoritmo genético

5.1 ordenacao_nao_dominada — separa em frentes

Classifica a população: frente 0 = não-dominados; frente 1 = os dominados só pela frente 0; e assim por diante.

```

for p in range(n):
    for q in range(n):
        if domina(objs[p], objs[q]): S[p].append(q) # p domina q
        elif domina(objs[q], objs[p]): cont[p] += 1 # alguem domina p
    if cont[p] == 0: frentes[0].append(p) # p e nao-dominado

```

5.2 distancia_multidao — mantém diversidade

Mede o quão “isolada” cada solução está. As das pontas recebem distância infinita (são sempre preservadas), as do meio recebem a distância entre os vizinhos. Isso evita que todas as soluções se amontoem num ponto só.

5.3 crossover_ox e mutacao_swap — gerar filhos

Cada indivíduo é uma **permutação** das VMs (uma ordem). O *Order Crossover* copia uma fatia de um pai e completa com a ordem do outro; a mutação troca duas posições. O laço principal (**nsga2**) repete: gera filhos, junta com os pais e mantém os **pop** melhores (por frente e diversidade).

Diferença para o VMPACS: o VMPACS *constrói* soluções com feromônio; o NSGA-II *evolui* uma população por cruzamento e mutação. Os dois buscam a mesma fronteira de Pareto, por caminhos diferentes.

6 referencia.py — a fronteira “melhor conhecida”

Serve para *visualizar* o trade-off real. Para cada número de servidores **k**, distribui as VMs entre os **k** servidores buscando equilíbrio, e refina com busca local.

```

for k in range(kmin, kmin + extra + 1): # varre numeros de servidores
    # varias tentativas: espalha as VMs entre k servidores, balanceando
    a = aloc_k_bins(rng.permutation(n), k, inst)
    # ... guarda a de menor desperdicio e refina com busca_local
    melhor = busca_local(melhor, inst, iters_local, rng)
return pontos_nao_dominados(np.array(solucoes))

```

busca_local tenta mover VMs entre servidores já usados para reduzir o desperdício total. Variando **k**, traça-se a curva de compromisso completa.

7 experimentos.py — rodar tudo

```
python3 experimentos.py run 0 # roda a correlacao P=0.0 (e salva parcial)
python3 experimentos.py run 1 # P=0.25 ... ate run 4 (P=1.0)
python3 experimentos.py agg # junta tudo: gera tabelas (CSV) e figuras (PNG)
```

Para cada correlação, ele roda VMPACS e NSGA-II 10 vezes, calcula as 4 métricas, e salva. O `agg` desenha os gráficos de barras e a fronteira de Pareto.

8 Como ler cada gráfico

Todos os gráficos comparam **VMPACS (vermelho)** e **NSGA-II (azul-escuro)** nos 5 níveis de correlação P (eixo horizontal; o número entre parênteses é o coeficiente de correlação aproximado, de $-0,75$ a $+0,75$). As barras têm *barras de erro* (o desvio entre as 10 execuções).

8.1 Gráficos de barras de energia, desperdício e servidores

- **Energia (W) e Desperdício e nº de Servidores:** em todos, **barra mais baixa = melhor**.
- Como ler: para cada P , compare as duas barras. A menor é o algoritmo que gastou menos / desperdiçou menos / usou menos servidores naquele cenário.
- O que esperar: o NSGA-II costuma ter barras um pouco menores (converge para soluções de menor energia e desperdício); em $P = 0,5$ o VMPACS empata ou ganha no desperdício.

8.2 Gráficos de Hipervolume e IGD

- **Hipervolume (HV):** barra mais **ALTA = melhor** (cobre mais área boa do espaço de objetivos). É a única em que “maior é melhor”.
- **IGD:** barra mais **BAIXA = melhor** (chegou mais perto da fronteira de referência).
- Como ler: HV alto e IGD baixo indicam o algoritmo que melhor se aproximou do ideal. Nas figuras, o NSGA-II tem HV maior e IGD menor na maioria das correlações.

8.3 Gráfico das fronteiras de Pareto

Este é o gráfico-chave do trabalho. **Eixos:** horizontal = energia (W); vertical = desperdício. **O canto inferior-esquerdo é o melhor** (pouca energia e pouco desperdício).

- A **curva cinza** (“fronteira melhor conhecida”) mostra o *trade-off real*: cada ponto é um compromisso (menos energia $\rightarrow$ mais desperdício, e vice-versa). Ela tem vários pontos justamente porque é a curva completa de melhores trocas.
- Os **marcadores** (quadrado vermelho = VMPACS; losango azul = NSGA-II) mostram *onde cada algoritmo converge*.
- Como ler: quanto mais para baixo e para a esquerda, melhor. O NSGA-II costuma “encostar” na curva (no joelho de menor energia); o VMPACS fica um pouco acima dela (minimiza servidores, mas equilibra um pouco menos).
- **Por que a curva é “rasa”?** Porque, nestas instâncias, os dois objetivos quase não conflitam: empacotar bem reduz energia e desperdício ao mesmo tempo. Por isso o trade-off existe, mas é pequeno — e cada algoritmo sozinho converge para perto de um único ponto (o joelho).

“Mas não deveria parecer uma exponencial?” Ótima pergunta — e a resposta é *sim*: uma fronteira de Pareto costuma ser uma curva decrescente e convexa (formato em “L”). A nossa *é* desse formato (Figura 1), com duas particularidades. (i) **É discreta**: a energia vale $\text{const} + 162 \cdot k$, com k inteiro igual ao número de servidores; logo a fronteira são *pontos* (de 162 W em 162 W), não uma curva contínua — as curvas lisas dos livros vêm de problemas *contínuos*. (ii) **É curta**: o desperdício *satura* após uns 2–3 servidores a mais que o mínimo (os servidores já ficam equilibrados), então poucos pontos são não-dominados. Vemos, assim, apenas a *parte inicial* daquela curva convexa.

A fronteira É decrescente/convexa — mas é DISCRETA (energia em degraus de 162 W por servidor) e RASA

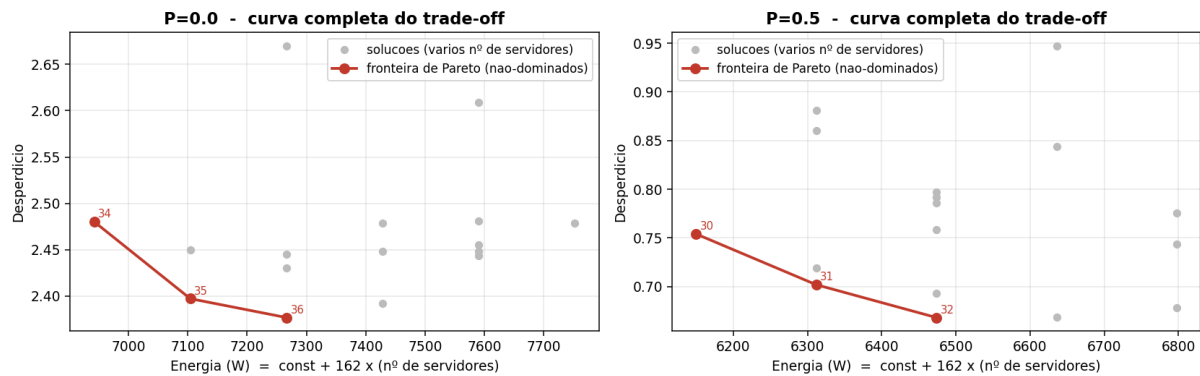


Figura 1: A fronteira de Pareto *é* decrescente e convexa (linha vermelha), mas discreta (degraus de 162 W por servidor) e curta (o desperdício satura). Pontos cinzas: soluções dominadas; rótulos: número de servidores.

Resumo de “o que é melhor” em cada gráfico: energia, desperdício, servidores, IGD e Spacing → **menor é melhor**; ONVG e Hipervolume → **maior é melhor**; nas fronteiras → **inferior-esquerda é melhor**.